



دانشکده مهندسی کامپیوتر

سیستم‌های عامل

فاز ۳ پروژه

نیم‌سال اول

سال تحصیلی ۱۴۰۴-۱۴۰۵

در انجام پروژه به نکات زیر توجه نمایید:

- برای تحویل پاسخ این پروژه لازم است در یک فایل zip تمام فایل هایی که تغییر دادید + یک گزارش مختصر از آنچه انجام داده‌اید را قرار بدید و یک فایل به نام OS_P3_student-id1_student-id2.zip در سامانه کورسز ارسال کنید (جزئیات بیشتر در بخش توضیحات پایانی توضیح داده شده است).
- کامنت گذاری در کد جهت افزایش خوانایی انجام شود و هر تکه از کد در بخش مخصوص خود نوشته شود.
- در صورت هر گونه مشکل می‌توانید به اکانت‌های تلگرام زیر پیام دهید:
@lkavehamdl و @Parsa22000
- در صورت کشف تقلب نمره ۰ برای تمام طرف‌های درگیر در نظر گرفته خواهد شد.
- مهلت انجام این پروژه تا ساعت ۲۳:۵۹ جمعه ۳۰ دی است.

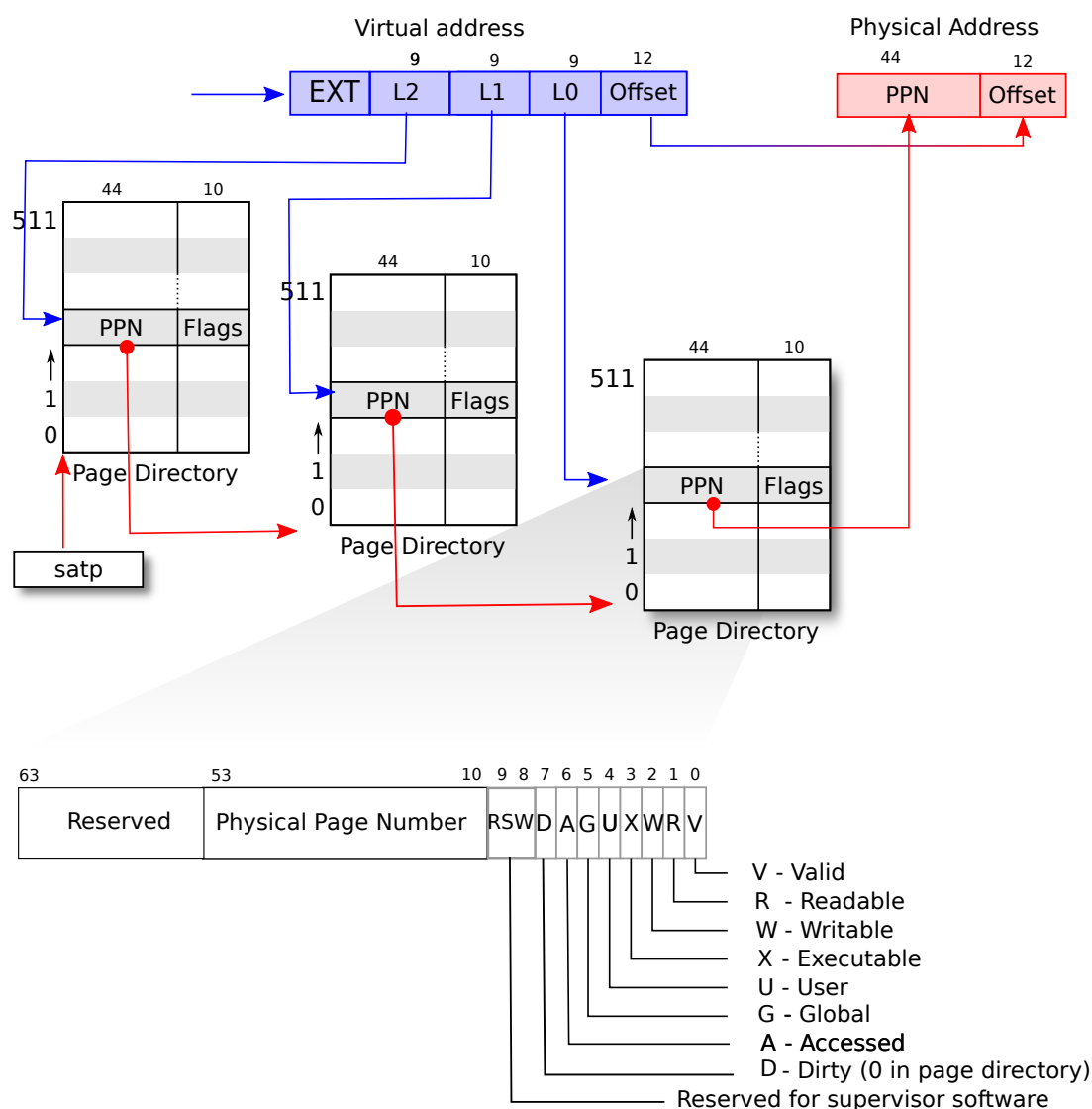
فهرست مطالب

۳	بخش اول: Swap Memory
۴	مرحله اول: پردازش در سطح kernel
۵	مرحله دوم: فرایند swap out
۵	مرحله سوم: فرایند swap in
۶	تست
۸	بخش دوم: Namespaces
۸	چیسـت Namesapce
۱۰	توضیحات هر Namespace
۱۲	نکات تکمیلی
۱۵	بخش امتیازی: syscall tracing
۱۸	توضیحات پایانی
۱۸	توضیحات
۱۸	آنچه که باید ارسال کنید

بخش اول: Swap Memory

در سیستم عامل لینوکس، هنگامی که یک پردازش درخواست حافظه می کند ولی جایی در حافظه اصلی موجود نیست، برخی از page های حافظه اصلی در حافظه فرعی ذخیره می شود و جای آن به آن پردازش داده می شود. به این بخش از حافظه فرعی Swap Memory و به این کار Swapping گفته می شود. در سیستم عامل xv6 این مکانیزم وجود ندارد و در این بخش قرار است آن را پیاده سازی کنیم.

در xv6 فرایند تبدیل آدرس مجازی به آدرس فیزیکی از طریق سه لایه page table انجام می‌شود. در شکل ۱ می‌توانید جزئیات آن را مشاهده کنید. برای اطلاعات بیشتر پیشنهاد می‌کنیم بخش **Xv6: Page tables** را مطالعه کنید.



شکل ۱: جزئیات page table های xv6

برای پیاده‌سازی swapping، هر page را در یک فایل به فرمت زیر ذخیره می‌کنیم:

```
<pid>_<VA[L2,L1,L0]>.swp
```

در این فرمت نام‌گذاری، <pid> شماره پردازش مربوط به آن page است. <VA[L2,L1,L0]> نیز بازه‌ای از آدرس مجازی است که آدرس‌های هر سه لایه page table را شامل می‌شود (از آنجایی که ۲۵ بیت پر ارزش آدرس‌های مجازی همیشه صفر است، تنها نیاز به بیت اندیس ۱۲ تا ۳۸ نیاز است).

توجه

اصولا پیشنهاد می‌شود در سطح kernel خواندن و نوشتن در فایل انجام نشود. برای حل این موضوع می‌توانید فرایند خواندن و نوشتن فایل را به یک پردازش در user space بسپارید. علت این امر در **این لینک** توضیح داده شده است.

در ادامه مراحل که برای پیاده‌سازی swapping نیاز است شرح داده شده است.

مرحله اول: پردازش در سطح kernel

از آنجایی که فرایند swapping می‌تواند زمان‌بر باشد، به جای اینکه به طور مستقیم انجام شود، به پردازش‌های در سطح kernel سپرده می‌شود. پردازش‌های kernel پردازش‌هایی هستند که تمام عمرشان در سطح kernel سپری می‌شود و نه user. برای ایجاد پردازش kernel، تابع زیر را تعریف کنید (name نام پردازش و entryptoint تابعی است که آن پردازش در سطح kernel اجرا می‌کند):

```
1 void create_kernel_process(const char *name, void (*entrypoint));
```

هنگامی که یک پردازش kernel ایجاد می‌شود، باید به صف مربوط به scheduler نیز اضافه شود. لذا نیاز است scheduler را به گونه‌ای تغییر دهید تا در صورت مواجهه با kernel process آن را اجرا کند.

مرحله دوم: فرایند swap out

هنگامی که پردازهای درخواست حافظه کرد و حافظه اصلی پر بود، نیاز به swap out کردن یکی از page های کمتر استفاده شده است (می‌توانید از سیاست LRU برای آن استفاده کنید). در ادامه این فرایند مرحله به مرحله توضیح داده شده است. ابتدا state پردازهای که درخواست حافظه کرده است به SLEEPING تغییر می‌کند. سپس به پردازه kernel که مسئولیت swap out را به عهده دارد، درخواست یک page آزاد داده می‌شود (این پردازه از قبل ایجاد شده است و صرفاً منتظر درخواست است). این پردازه به هنگام دریافت این درخواست، اطلاعات مربوط به یکی از page های کمتر استفاده شده را به همان فرمتی که پیش‌تر گفته شد، ذخیره می‌کند. سپس بیت Valid در PTE آن page را صفر می‌کند تا آن page به عنوان page آزاد تلقی شود. در نهایت بیت Swapped Out آن page را یک می‌کند تا در آینده امکان بازیابی آن page وجود داشته باشد.

توجه

از آنجایی که نیاز به بیت Swapped Out در هر page است، آن را باید به صورت دستی تعریف کنید (مانند آنچه در COWfork فاز ۲ انجام دادید).

در هنگام پیاده‌سازی نکات زیر را رعایت کنید:

- پردازه swap out باید درخواست‌ها را از یک صف بخواند.
- در صورتی که درخواستی برای اجرا نبود، این پردازه باید از اجرا توسط scheduler منع شود (می‌توانید اینکار را با تغییر state آن به SLEEPING انجام دهید).
- در صورتی که حداقل یک page فیزیکی خالی شود، تمام پردازه‌هایی که به دلیل نبود فضای حافظه SLEEPING شده بودند باید بیدار شوند.

مرحله سوم: فرایند swap in

هنگامی که پردازهای که swap out می‌شود، چون دیگر page آن در page table نیست، تنها یک عملیات حافظه کافیسیت تا page fault رخ دهد. زمانی که kernel

این page fault را تشخیص می‌دهد، باید علت آن را بررسی کند. اگر بیت Swapped در Out PTE آن page table یک باشد، یعنی علت آن swapping بوده است. در این صورت پردازهای که موجب page fault شده است باید RUNNING شود. در این هنگام به پردازه kernel که مربوط به swap in است درخواست بازگردانی آن page داده می‌شود. زمانی که این پردازه درخواستی را دریافت می‌کند، یک page فیزیکی را allocate می‌کند، آن را با محتوای swap شده آن page پر می‌کند و در نهایت در آن به آدرس مجازی داده شده نگاشت می‌کند.

در هنگام پیاده‌سازی نکات زیر را رعایت کنید:

- در صورتی که علت page fault چیزی غیر از swap out باشد، جلوی آن پردازه باید گرفته شود (مانند آنچه اکنون رخ می‌دهد).
- پردازه swap in باید درخواست‌ها را از یک صف بخواند.
- در صورتی که درخواستی برای اجرا نبود، این پردازه باید از اجرا توسط scheduler منع شود (می‌توانید اینکار را با تغییر state آن به SLEEPING انجام دهید).
- زمانی که کار پردازه swap in تمام شد، باید پردازهای که موجب آن شده بود را بیدار کند.

توجه

- برای پیاده‌سازی این مرحله ممکن است نیاز به تغییر struct proc پیدا کنید.
- در پیاده‌سازی حواستان به مشکلات synchronization احتمالی باشد.

تست

برای تست کردن صحت کارکرد این مرحله، برنامه‌ای تعریف کنید که ۲۰ فرزند تولید می‌کند و هر فرزند در یک حلقه با ۱۰ مرتبه تکرار، 4KB فضا را با malloc اشغال می‌کند. بعد از هربار اشغال حافظه، آن را با یک تابع دلخواه پر کنید و در نهایت با همان تابع صحت داده‌های موجود در آن فضا را بررسی کنید.

توجه

برای تست ساده‌تر، می‌توانید مقدار PHYSTOP واقع در فایل `kernel/memlayout.h` را کاهش دهید. این کار باعث می‌شود محدوده حافظه‌ای که برای پرده‌های سطح user تخصیص پیدا می‌کند کم باشد و مکانیزم swapping رخ دهد.

بخش دوم: Namespaces

در درس رایانش ابری با چگونگی کار کردن docker آشنا خواهید شد. که در واقع یک docker container تشکیل شده از دو عنصر کلیدی در Linux تشکیل شده است:

- Cgroups

- Namespaces

در این بخش می‌خوایم در مورد Namespace ها بیشتر صحبت کنیم و چند مورد از آنها رو پیاده سازی کنیم:

چیست Namesapce

به صورت کلی Namespace یک حالت انتزاعی از kernel است که منابع کار آن مجازی سازی منابع موجود است.

بجای استفاده از منابعی مانند filesystem به صورت global هر پردازش از طریق Namespace مربوطه می‌تواند منابع مجاز را ببیند و ارتباط مستقیمی با منابع ندارد. دو پردازش می‌توانند دید مختلفی به سیستم عامل داشته باشند با وجود این که در یک kernel مشترک اجرا می‌شوند.

در این بخش از شما می‌خواهیم که ۴ Namespace اصلی که Linux هم استفاده شده است را پیاده سازی کنید:

PID •

Mount •

UTS •

IPC •

که در ادامه هر یک توضیح داده می‌شوند.
ولی اول باید یک Design Pattern از Namespace ها ببینیم:
هر Namespace یک Object دارد که به شکل زیر است (به عنوان مثال برای PID):

```
1 struct pid_namespace{
2     int refcnt;
3     int nex_pid;
4     struct proc* init;
5 };
```

و در هر پردازش یک pointer به آن وجود خواهد داشت:

```
1 struct proc{
2     ...
3     struct pid_namespace* pid_ns;
4 };
```

این Namespace ها تا زمانی معتبر هستند که حداقل یک پردازش به آن ها اشاره‌گری داشته باشد.

در زمان `fork()` کردن پردازش ها می‌توانند که namespace ها را یا به اشتراک بگذارند و یا پردازش فرزند برای خود namespace جدیدی ایجاد کند. که در واقع شروع مفهوم `clone()` است.

برای پیاده سازی این پروژه شما باید اقدامات زیر را انجام دهید:

- namespace های عنوان شده را به kernel اضافه کنید

- ساختاری struct proc را تغییر بدهید

- پیاده سازی syscalls namespace-aware

- تغییر دادن در fork()

- نوشتن فایل تست در user space

توضیحات هر Namespace

PID

این namespace هر پردازش یک pid counter مجزا برای خود دارد و در هر namespace ای $pid = 1$ وجود دارد. پردازش والد می تواند namespace پردازش فرزند را ببیند ولی برعکس آن مقدور نبوده.

ساختاری struct مربوطه در کرنل باید به شکل زیر باشد:

```
1 struct pid_namespace {
2     int refcount;
3     int next_pid;
4 };
```

و مطابق توضیحات فوق در struct proc اضافه شود.

Mount

هر پردازنده باید بتواند `root("/")` خود را ببیند و از mount table خود استفاده کند. ساختار struct مربوطه به شکل زیر خواهد بود:

```
1 struct mount_namespace {
2     int refcnt;
3     struct inode *root;
4 };
```

و مطابق توضیحات داده شده باید به struct proc اضافه شود.

UTS(Hostname)

وظیفه اصلی این namespace، isolate کردن hostname است. برای این بخش شما نیاز به پیاده سازی دو syscall به نام های `getHostname()` و `setHostname()` دارید چرا که در خود Xv6 پیاده سازی نشده است. ساختار struct مربوطه به شکل زیر خواهد بود:

```
1 struct mount_namespace {
2     int refcnt;
3     char hostname[64];
4 };
```

و مطابق توضیحات داده شده باید به struct proc اضافه شود.

IPC

در سیستم های Linux این namespace وظیفه isolate کردن موارد زیر را دارد:

- pipes
- shared memory
- message queue

که در XV6 فقط pipe وجود دارد. فلذا ساختار struct مربوطه به شکل زیر خواهد بود:

```
1 struct mount_namespace {  
2     int refcnt;  
3 };
```

و مطابق توضیحات داده شده باید به struct proc اضافه شود.

نکات تکمیلی

باید حواستان به Lifecycle Namespace باشد. و هر زمان مقدار refcnt در هر namespace برابر با ۰ شود باید حافظه مربوط به آن آزاد شود. برای انجام این کار نیاز به ایجاد دو syscall برای کم و زیاد کردن مقدار refcnt ها دارید:

افزایش:

```
1 void ns_incref(struct ns *ns) {
2     ns->refcount++;
3 }
```

کاهش:

```
1 void ns_decref(struct ns *ns) {
2     ns->refcount--;
3     if (ns->refcount == 0) {
4         free(ns);
5     }
6 }
```

توجه

از syscall اولی باید در fork() و از syscall دوم باید در exit() صدا زده شوند.

همچنین شما باید یک API به نام unshare ایجاد کنید که به پردازش اجازه ایجاد namespace بدهد. ورودی این API یک bitmask باشد که نشان دهنده flag ها است:

```
1 int unshare(int flags);
```

Flag	Meaning
CLONE_NEWPID	Create a new PID namespace for this process
CLONE_NEWNS	Create a new mount (filesystem) namespace
CLONE_NEWUTS	Create a new UTS namespace (hostname/domain)
CLONE_NEWIPC	Create a new IPC namespace (for pipes, if used)

پس باید در سمت kernel هم یک `syscall` اضافه شود که از reference های namespace های قبلی کم بشود و به پردازش یک namespace جدید assign شود.

بخش امتیازی: syscall tracing

ضمن عرض خستنه نباشید تا به اینجای کار خدمت دانشجویان. تا به اینجا شما syscall اضافه کردید، روند scheduler رو تغییر دادید، با memory کار کردید (بیشتر هم قراره کار بکنید در بخش بعد) ولی تا الان به kernel به دید یک black box نگاه کردید. در این بخش می‌خواهیم تا بیشتر وارد kernel بشویم. در سیستم عامل دو فضای user و kernel وجود دارد. syscall ها در واقع پل ارتباطی بین untrusted userspace و trusted kernelspace هستند. پس در واقع نیاز داریم که به syscall ها فراتر از رفتار نگاه کنیم تا بتوانیم خود OS رو دیباگ کنیم. پس در واقع شما نیاز دارید یک syscall به شکل زیر پیاده سازی کنید:

```
1 int trace(int mask);
```

برای عملکرد بهتر باید از bitmask به عنوان ورودی استفاده شود و نه عدد صحیح. به عنوان مثال کد صفحه بعد را برای تست کردن trace پیاده سازی کنید. در نشر داشته باشید bit هایی که در این bitmask روشن می‌کنید در واقع مقادیری است که در فایل syscall.h تعریف می‌کنید. در نظر داشته باشید که کد شما باید بتواند از multiple selective tracing پشتیبانی کند. همچنین در صورتی که ورودی trace مقدار ۱- باشد، باید تمامی syscall ها را trace کرد و در صورت ورودی ۰ هیچ syscall ای را نباید trace کرد. همچنین در هنگام fork کردن یکی از موادی که پردازش فرزند از والد به ارث می‌برد، trace bitmask است.


```

1  #include "kernel/types.h"
2  #include "kernel/syscall.h"
3  #include "user/user.h"
4
5  int
6  main(void)
7  {
8      int pid;
9
10     printf("=== TRACE TEST START ===\n");
11
12     // Test 1: trace write only
13     trace(1 << SYS_write);
14     write(1, "A\n", 2);
15
16     // Test 2: mask filtering
17     open("README", 0);
18
19     // Test 3: multiple syscalls (write + exit)
20     trace((1 << SYS_write) | (1 << SYS_exit));
21     write(1, "B\n", 2);
22
23     // Test 4: fork inheritance
24     trace(1 << SYS_write);
25     pid = fork();
26
27     if (pid == 0) {
28         write(1, "child\n", 6);
29         exit(0);
30     } else {
31         write(1, "parent\n", 7);
32         wait(0);
33     }
34
35     // Test 5: exec tracing
36     trace(1 << SYS_exec);
37     pid = fork();
38
39     if (pid == 0) {
40         char *argv[] = { "echo", "exec-ok", 0 };
41         exec("echo", argv);
42         exit(1); // should not happen
43     } else {
44         wait(0);
45     }
46
47     // Test 6: disable tracing
48     trace(0);
49     write(1, "NO TRACE\n", 9);
50
51     printf("=== TRACE TEST END ===\n");
52
53     exit(0);
54 }

```

خروجی سمت user:

```
1  === TRACE TEST START ===
2  A
3  B
4  parent
5  child
6  exec-ok
7  NO TRACE
8  === TRACE TEST END ===
```

خروجی kernel

```
1  < pid >: syscall write -> 2
2  < pid >: syscall write -> 2
3  < pid >: syscall write -> 7
4  < pid2 >: syscall write -> 6
5  < pid >: syscall exec -> 0
```

توضیحات پایانی

از شما درخواست داریم که یک private repository در گیت‌هاب درست کنید و تغییرات کد خود را مرحله به مرحله commit کنید و در صورت تمایل می‌توانید هر یک از تدریس‌یاران را به پروژه خود اضافه کنید. دقت کنید که شما نبایستی برنامه‌های خود را با دیگر دانشجویان به اشتراک بگذارید.

آیدی گیت‌هاب ما:

کاوه: lkavehamdl

پارسا: Parsa2201

توضیحات

- این فاز پیش‌نیاز قطعی فازهای بعدی است و انجام ندادن آن باعث می‌شود که نتوانید فاز دوم را شروع کنید. و همچنین نمی‌توانید برای انجام پروژه گروهی را تشکیل دهید.
- پروژه شما تحویل آنلاین خواهد داشت بنابراین از استفاده از کدهای دیگران یا کدهای موجود در وب که قادر به توضیح دادن عملکرد آنها نیستید، بپرهیزید.
- ابهامات خود را در گروه درس در تلگرام مطرح کنید و ما در سریع‌ترین زمان ممکن به آنها پاسخ خواهیم داد.

آنچه که باید ارسال کنید

یک فایل zip با نام OS_P1_student-id.zip (student-id را با شماره دانشجویی خود جایگزین کنید) که شامل دو مورد زیر است:

- گزارش خیلی مختصر از آنچه که انجام داده‌اید تا دو فراخوانی سیستمی خواسته شده را به xv6 اضافه کنید.
- پوشه‌ای که در آن کدهای شما وجود دارد. دقت کنید که تنها و تنها فایل‌هایی را که تغییر داده‌اید یا اضافه کرده‌اید را برای ما بفرستید.